



Palestine Technical University - Kadoorie

College of Information Technology and Artificial Intelligence

Computer Science Department

Theory of Computation

Advanced Vending Machine Project

Authors : Layth Hersheh, Anas Maali, Amr Omair, Israa Shtewi

Course: Theory of Computation

December 2025

Table of Contents

Contents

Table of Contents	2
1 Abstract	3
2 Introduction	3
2.1 Problem Statement	3
2.2 Project Overview	3
2.3 Group Formation	4
2.4 Key Deadlines	4
3 Literature Review	5
3.1 Vending Machine Systems	5
3.2 Mobile-to-Machine Interaction	5
3.3 Theoretical Foundations	5
4 Methodology	5
4.1 Overview	6
4.2 Requirements and Use Cases	6
4.3 Formal Definition of the Finite Automaton (Finite Automaton)	6
4.4 Formal Finite Automaton (FA) Definition:	9
4.5 High-Level Design	10
4.5.1 User → Website (Frontend State Controller)	10
4.5.2 Website → Backend Server	11
4.5.3 Backend Server → Vending Machine	12
4.5.4 Vending Machine → Database + Backend	12
4.5.5 Database → Backend → Website → User	13
5 Code Implementation	13
5.1 Code Implementation Overview	13
5.1.1 Session Tracker (State Controller)	13
5.1.2 Event-Driven Actions	13
5.1.3 Validation and Guarding	13
5.1.4 Linear Transaction Flow	14
5.1.5 Practical vs Theoretical	14

1 Abstract

This project presents the design and implementation of an advanced vending machine system that integrates mobile interaction through QR code scanning. The system addresses the limitations of traditional vending machines by introducing a modern, user-friendly interface and a structured control model based on computational theory. A core contribution of this work is the formal modeling of the vending machine's behavior using Finite Automata, enabling precise state management and predictable system transitions. The solution includes a mobile-to-machine communication flow, an event-driven architecture, and a state-based software design that accurately reflects real-world machine behavior. The results demonstrate a fully functional prototype capable of showing state changes, processing transactions, and completing successful dispensing sequences. This project highlights the practical application of Theory of Computation concepts in real engineering systems and provides a foundation for future enhancements in automated retail technologies.

2 Introduction

2.1 Problem Statement

Traditional vending machines operate through a fixed hardware interface that relies on physical buttons and coin or bill inputs. These systems often suffer from usability limitations, lack of flexibility, and minimal interaction feedback, making them prone to user errors and operational inefficiencies. Additionally, traditional machines do not support modern digital payment methods, and their rigid interface design makes it difficult to extend or upgrade functionality. To address these challenges, this project proposes the integration of a mobile application as the primary user interface. By scanning a QR code displayed on the vending machine, the user can securely initiate the transaction process using their mobile device. This approach enables a more intuitive interaction flow, supports a wider range of inputs, and allows future extensions such as digital payments or remote monitoring. From a theoretical perspective, the system's behavior can be formally modeled using

concepts from the Theory of Computation, particularly Finite Automata. These models enable a structured representation of the vending machine's states, transitions, and accepted sequences of user actions. Applying computational theory ensures that the system's logic is deterministic, predictable, and aligned with formal language principles taught in the course.

2.2 Project Overview

This project presents the design and implementation of an advanced vending machine system that utilizes a mobile application as the main interface for user interaction. The user begins the process by scanning a QR code displayed on the machine, which immediately establishes a communication channel between the application and the system. Through this connection, the user can select an item, initiate payment, and observe real-time feedback from the machine.

The vending machine's internal logic is driven by a Finite Automaton (FA) that governs all state transitions, including waiting for a QR scan, processing payment inputs, dispensing the selected item, and completing the transaction. The FA ensures that the system behaves consistently and that all events follow a valid sequence according to the formally defined language of the system. This integration of theory and practice results in a robust, predictable vending system that demonstrates the real-world application of computation theory.

2.3 Group Formation

This project was completed by a group of four students:

- Layth Hersheh
- Anas Maali
- Amr Omair
- Israa Shtaiwy

The group was officially registered through the required Group Registration Form, as instructed by the course guidelines.

2.4 Key Deadlines

Table 1: Project Key Deadlines

	Table 1: Project Key Deadlines	
	Milestone	Date
gray!80!black	Project Report Submission Deadline	Monday, 15/12/2025
	Project Discussion Day	Saturday, 20/12/2025

3 Literature Review

3.1 Vending Machine Systems

Overview of traditional vending machine mechanisms. Traditional vending machines operate through a combination of mechanical components and embedded control units. These machines typically detect inserted coins, validate currency, and activate dispensing mechanisms based on predefined logic. Early systems relied heavily on analog circuits, but modern machines now use microcontrollers, sensors, and digital payment modules to ensure accuracy, reliability, and security.

Common computational models used in vending systems. Vending machine logic is often modeled using computational frameworks such as Finite State Machines (FSMs). These models represent the machine's behavior as transitions between states—such as “idle,” “waiting for input,” “validating payment,” and “dispensing.” DFA (Deterministic Finite Automata) is widely used to design cash-handling logic, ensuring that each user input leads to a predictable system response.

3.2 Mobile-to-Machine Interaction

QR code technology. QR codes enable fast, secure, and user-friendly communication between mobile devices and physical machines. By scanning a QR code, the user can access product information, make selections, or initiate payment processes. QR-based systems provide a bridge between cloud services and hardware devices.

Basics of client–machine communication. Client-to-machine interaction commonly occurs through REST APIs, Bluetooth Low Energy (BLE), or local network protocols. The mobile device acts as the client, sending requests such as item selection or payment confirmation, while the machine serves as the server, responding with system status or executing operations.

Event-driven interaction in mobile-integrated systems. Mobile-integrated vending systems rely on event-driven architectures. Events—such as scanning a code, selecting a product, or completing a payment—trigger discrete system actions. This approach reduces latency, improves synchronization between mobile and machine processes, and enhances the overall user experience.

3.3 Theoretical Foundations

Finite Automata (DFA/NFA). Finite Automata provide a mathematical model for designing system logic. DFAs are particularly useful for implementing deterministic and predictable vending workflows, while NFAs offer flexibility in modeling more complex or ambiguous input patterns.

Regular Languages. Regular languages define the class of languages recognized by finite automata. In vending machines, they help describe and validate sequences of inputs—such as coin insertion patterns or payment commands.

State Machines in software engineering. State machines are essential tools for modeling reactive systems. They define how the system transitions between states in response to external inputs. In vending systems, state machines model the logical flow from product selection to payment verification and dispensing.

(Optional Bonus) Moore and Mealy Machine concepts. Moore and Mealy machines are two variants of FSMs. In Moore machines, outputs depend only on the current state. In Mealy machines, outputs depend on both the current state and input. These models help optimize vending workflows—Mealy for real-time responsiveness and Moore for simpler, more stable design.

4 Methodology

4.1 Overview

This section explains how the qr-enabled vending machine system was designed and implemented step by step. The methodology follows logical sequence that start from identifying the system requirements, then modeling the system using concepts from the Theory of Computation, and finally developing, integrating, and testing the full application. Each design choice such as using a deterministic finite automaton (DFA) OR ENFORCING atomic transactions is directly connected to ensuring predictable and reliable system behavior. By combining formal modeling with practical development, the final solution remains both theoretically sound and technically functional.

4.2 Requirements and Use Cases

Functional Requirements:

- The system must allow a user to scan a QR code placed on the vending machine using their mobile device.
- After scanning, the user should be able to view the available items, choose one, and complete a simulated payment.
- The vending machine must be able to update and expose its internal state (such as current mode, selected item, and transaction progress).

Non-Functional Requirements:

- Transactions must be atomic and recoverable, meaning the system should either complete the entire purchase successfully or return to the previous safe state if something goes wrong (e.g., payment failure, disconnection).
- The finite automaton controlling the machine must be a deterministic FA (DFA) to guarantee predictable, one-path behavior for every input and avoid ambiguity during state transitions.

Primary Use Cases:

- An anonymous user scans the QR code and gets directed to registration process then proceeds to view the list of items in the machine.
- A logged-in user selects an item, completes payment, and the machine dispenses the product
- if a transaction fails at any point either a item error or a payment error, the system rolls back steps and returns to a wanted predefined state depending on the error.

4.3 Formal Definition of the Finite Automaton (Finite Automaton)

In this step, the vending machine system is modeled as a finite automaton(FA). This allows us to formally represent all possible system states and transitions, ensuring predictable behavior.

DFA Diagram

The figure below shows the DFA representation of the vending machine system.

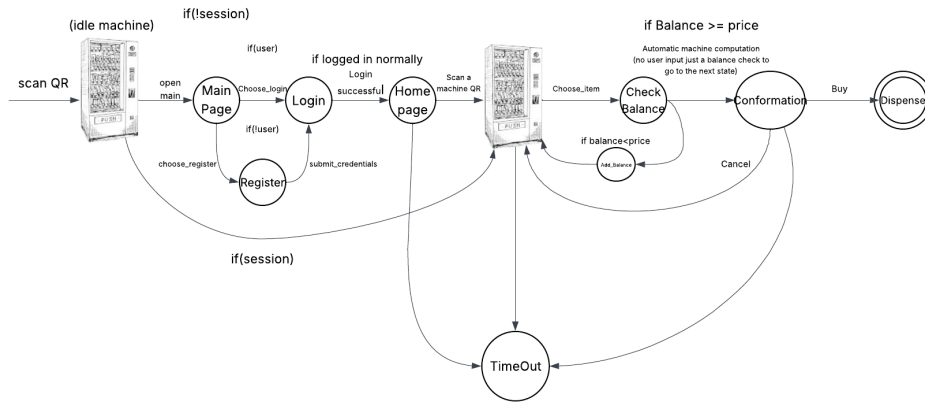


Figure 1: Deterministic Finite Automaton (DFA) for the vending machine.

Identifying States (Q)

The machine can exist in several discrete phases, each representing a real-world step in the user interaction:

START The initial idle state where the machine waits for a user to scan the QR code. No actions have been taken yet.

MainMenu The main page displayed after scanning the QR code. Here, the user can decide whether to log in or register a new account. The choice determines the next state.

LoginForm The login page/form where existing users enter their credentials to access their account.

RegisterForm The registration page/form for new users to create an account, providing the necessary details.

LoggedInHome The home page for authenticated users, accessible after successful login or registration. From here, users can proceed to select items from the vending machine.

ItemSelection The state where users choose one or multiple items they want to purchase. Each item selection updates the current cart total.

CheckBalance The machine verifies if the user's account balance is sufficient to cover the total price of the selected items.

AddMoreBalance If the balance is insufficient, the user can add more money. The machine then returns to *CheckBalance* to re-evaluate the total.

ProceedPrompt The confirmation state where the user decides to either buy the selected items (*buy*) or cancel the transaction (*cancel*).

Dispensing The machine is actively dispensing the purchased items. The user cannot make further inputs in this state.

END The final state indicating a successful transaction after dispensing is complete. The session ends here.

Error Any unexpected failure, such as session timeout, payment error, or system malfunction, leads to this state. It is a dead state where the transaction cannot proceed.

Defining Alphabet (Σ)

The alphabet represents all possible input events that trigger transitions between states:

- **scanQR** — User scans the QR code on the vending machine to start.
- **openMain** — The main page opens after scanning the QR, showing options like login or register.
- **choose_login** — The user clicks the login button to enter their existing account credentials.
- **choose_register** — The user clicks the register button to create a new account.
- **submit_credentials** — The user submits the login or registration form with their credentials.
- **scanMachineQR** — The user scans a vending machine QR if separate from the initial QR step.
- **choose_item** — The user selects one or multiple items to purchase from the machine.
- **done_select** — The user finishes selecting items and is ready to proceed to payment or balance check.
- **add_balance** — The user adds more money to their account if the current balance is insufficient.
- **buy** — The user confirms the purchase of the selected items.
- **cancel** — The user cancels the current transaction or action at any point before dispensing.
- **timeout** — The session times out or an unexpected error occurs, ending the transaction.

Language Specification: Regular Expression or CFG

```
L = ScanQR OpenMenu (ChooseLogin | ChooseRegister) SubmitCredentials  
  (ScanQR)?  
  (Choose_item)*  
  (add_balance)? (Buy | cancel)  
  dispense
```

Our vending machine design follows a straight-line, linear path, where the user moves step by step to complete a purchase. This demonstrates that the process is linear and sequential, without any branching, nesting, or recursion. Regular Expressions (REs) are perfect for modeling our design because they efficiently capture this linear structure of event sequences. In contrast, Context-Free Grammars (CFGs) are intended for hierarchical or nested recursive structures, which our machine intentionally lacks. We designed our vending machine to be simple, reliable, and fully deterministic, making REs the ideal choice for describing its behavior.

Limitations Faced with FA

1. Uncertainty between NFA and DFA

At the beginning, we were not sure whether the machine should be modeled as an NFA or a DFA. The process felt confusing because the system originally behaved in a nondeterministic way, but the final formal model had to be deterministic.

2. Handmade and unknown event language

Our event language (Σ) was not based on any standard. We had to invent the inputs ourselves, which made it difficult to decide what counts as an event between two states.

3. The language can grow very large

Depending on the vending machine's design, the number of possible events and valid sequences grows quickly. Without prior knowledge of the machine's full behavior, the language can become huge and hard to manage.

4. State explosion

As we added login, item selection, balance top-ups, and QR scanning, the number of states increased rapidly. The diagram became harder to draw and understand.

5. Difficulty ensuring full determinism

Ensuring every state had exactly one valid transition per event was challenging. We also had to make sure the FA, Language L , and the app flow stayed consistent. Sometimes the FA allowed sequences that the UI did not allow, and fixing that caused confusion.

6. No standard naming or structure for events

Since there is no common “vending machine event language,” we had to create our own naming system and transition rules. This made it difficult to stay organized and keep everything readable.

7. Modeling cancellation, timeout, or parallel actions

Our vending machine follows a simple linear flow, but the real app has features like timeouts and cancellation at different moments. These are difficult to model in a DFA because DFAs handle only one event at a time and have no concept of time or parallel behavior.

8. App implementation was sometimes easier than FA design

In some cases, the actual app behavior (event $\rightarrow$ event transitions) was easier to implement than to convert into a DFA. Designing the DFA felt harder because finite automata have strict limitations that do not always match how high-level applications behave.

Why the Vending Machine Language is Regular

The vending machine language is considered a **regular language** for several reasons:

1. It is designed using a Finite Automaton (FA).

Any language accepted by a DFA or NFA is, by definition, a regular language.

2. It consists of a finite sequence of events.

The inputs such as `scanQR`, `openMain`, `choose_login`, `submit_credentials`, and `choose_item` all come from a finite event alphabet (Σ). The machine moves linearly from one event to the next.

3. There is no recursion or nested structure.

The flow of actions does not include nesting, recursion, or memory-based repetition. The system depends only on the current state, which is a key property of regular languages.

4. It can be expressed as a Regular Expression (RE).

Because the system is linear and finite, we can write a regular expression that describes all valid transaction sequences. For example:

```
L = ScanQR OpenMenu (ChooseLogin | ChooseRegister) SubmitCredentials
    (ScanQR)?
    (Choose_item)*
    (add_balance)? (Buy | cancel)
    dispense
```

Being expressible with an RE confirms that the language is regular.

4.4 Formal Finite Automaton (FA) Definition:

Component	Definition
Q	{START, MainMenu, LoginForm, RegisterForm, LoggedInHome, ItemSelection, CheckBalance, AddMoreBalance, ProceedPrompt, Dispensing, TimeOut, Error, END}
Σ	{scanQR, choose_login, choose_register, submit_credentials_ok, submit_credentials_fail, choose_item, done_select, balance_ge_price, balance_lt_price, add_balance, buy, cancel, dispense_complete, timeout, session_end}
black	δ (START, scanQR) → MainMenu (MainMenu, choose_login) → LoginForm (MainMenu, choose_register) → RegisterForm (LoginForm, submit_credentials_ok) → LoggedInHome (LoginForm, submit_credentials_fail) → LoginForm (RegisterForm, submit_credentials_ok) → LoggedInHome (RegisterForm, submit_credentials_fail) → RegisterForm (LoggedInHome, choose_item) → ItemSelection (LoggedInHome, timeout) → LoggedInHome (ItemSelection, choose_item) → ItemSelection (ItemSelection, done_select) → CheckBalance (CheckBalance, balance_ge_price) → ProceedPrompt (CheckBalance, balance_lt_price) → AddMoreBalance (AddMoreBalance, add_balance) → CheckBalance (ProceedPrompt, buy) → Dispensing (ProceedPrompt, cancel) → LoggedInHome (Dispensing, dispense_complete) → END (Dispensing, timeout) → Final State (LoggedInHome, session_end) → TimeOut (ItemSelection, session_end) → TimeOut (CheckBalance, session_end) → TimeOut (ProceedPrompt, session_end) → TimeOut (Dispensing, session_end) → TimeOut (Time Out , session_end) → Dead State

Table 2: DFA 5-tuple representation

4.5 High-Level Design

4.5.1 User → Website (Frontend State Controller)

The user interacts with the system through the website using their mobile device. The frontend interface is responsible for the following tasks:

- Displaying the UI screens such as *Login*, *Register*, *Home*, *Select Item*, and *Confirmation*.
- Running the DFA state logic on the frontend (e.g., transitions such as *Login* → *Home*).
- Scanning the vending machine QR code to initiate the session.

Relation to the DFA:

- These UI screens directly correspond to the DFA states:
 - Main Page
 - LoginForm / RegisterForm
 - LoggedInHome
 - ItemSelection
 - ProceedPrompt
- Every user action on the frontend is treated as an *input event* to the DFA, triggering a state transition.

4.5.2 Website → Backend Server

Once the user interacts with the frontend, the website sends structured requests to the backend. The backend is responsible for the following components:

- **Authentication System**
 - Matches DFA transitions such as:
 - `submit_credentials(ok)` → *LoggedInHome*
 - `submit_credentials(fail)` → *LoginForm*
- **Inventory Management System**
 - Handles item-related actions such as:
 - *choose_item*
 - *CheckBalance*
 - *AddMoreBalance*
- **Payment Handler**
 - Activated when the user confirms a purchase:
 - `buy` → *Dispensing* (initiates payment and deducts balance)
- **DFA State Controller (Backend-side)**
 - Ensures the user cannot bypass states illegally.
 - Even if the frontend attempts invalid transitions, the backend enforces the correct DFA flow.

Relation to the DFA:

- The backend acts as the guardian of valid DFA transitions.
- For each incoming request, the backend verifies:
 - Is this transition allowed by the DFA?
- If yes → the backend continues processing the request.
- If no → the backend returns an error or forces a *Session Timeout*.

4.5.3 Backend Server → Vending Machine

When all conditions are satisfied (balance greater than or equal to price and the user confirms the purchase), the backend sends a machine command such as:

- “Dispense item X”
- “Display QR for next user”
- “Report machine status”

Machine Responsibilities:

- **Machine controller** → executes the dispensing process.
- **QR code display** → used to initiate new user sessions.
- **Local state monitor** → reports status to the backend (timeouts, errors, stock levels).

Relation to the DFA:

- Matches the transitions:
 - *ProceedPrompt (buy)* → *Dispensing*
 - *Dispensing* → *END*
- The vending machine completes the final DFA state transition.

4.5.4 Vending Machine → Database + Backend

After the vending machine dispenses an item:

- Stock is updated
- Transaction is logged
- User balance is updated

Database stores:

- Users
- Passwords
- Balances
- Item stock
- Prices
- Transactions

Relation to the DFA:

- The final transaction corresponds to the *END* state in the DFA.

4.5.5 Database → Backend → Website → User

All updated information is sent back to the user, including:

- New balance
- Updated stock
- Transaction confirmation

This completes the user session.

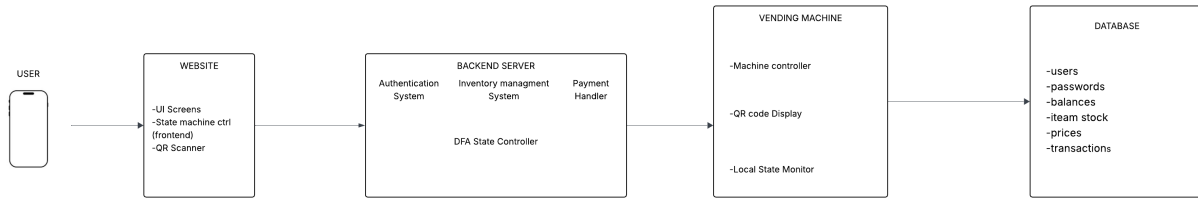


Figure 2: High level design

5 Code Implementation

The vending machine system is built in **Laravel** using two main controllers: **AuthController** and **VendingMachineController**. **AuthController** handles user registration and login, while **VendingMachineController** manages scanning the machine QR, adding items to the cart, checking balance, and completing purchases. Each user's session keeps track of their progress, so the system reacts to actions like QR scanning, item selection, or buying. Although the code does not use a formal State Pattern, the workflow behaves like a state machine, moving logically through phases such as:

`Idle` → `Authenticated` → `ItemChosen` → `PaymentProcessing` → `Dispense` → `TransactionComplete`

Canceling or timing out returns the system to `Idle`. This design mirrors a finite automaton in theory, making the vending machine language regular.

5.1 Code Implementation Overview

5.1.1 Session Tracker (State Controller)

This controller uses session data to track the user's current state and can display it in a graphical way, effectively showing what phase of the transaction the user is in.

5.1.2 Event-Driven Actions

Actions like scanning the QR code, logging in, adding items to the cart, or buying are treated as **events**. Each event moves the user from one phase to another depending on the input.

5.1.3 Validation and Guarding

The code checks inputs at every step, starting from registration and login credentials to item selection and payment. It is designed to handle errors gracefully, preventing the system from reaching an invalid state except in cases like session timeout.

5.1.4 Linear Transaction Flow

The system follows a clear, linear flow: `login` → `choose items` → `check balance` → `buy` → `dispense` → `transaction complete`. There is no recursion or nested loops, which aligns with a regular language model.

5.1.5 Practical vs Theoretical

While the Laravel implementation is more practical and manageable than a textbook finite automaton, the workflow theoretically behaves like a finite automaton, which is the key point for the **Theory of Computation** part of this project.